



TEORÍA DE ALGORITMOS
CURSO ECHEVARRIA

Trabajo Práctico 1

June 2, 2025

- Juan Claros - 111254
- Yoel Gaston Arlia - 111520
- Guido Cuppari - 111172
- Josue Daniel Mamani - 111514

Índice

1	<i>Problema 1</i>	3
1.1	Enunciado y Supuestos	3
1.1.1	Enunciado	3
1.1.2	Supuestos	3
1.2	Diseño	3
1.3	Pseudocódigo	4
1.4	Análisis de Complejidad	5
1.5	Seguimiento	6
1.6	Casos de prueba y mediciones	8
1.7	Resultados	8
1.8	Conclusiones	9
1.9	Generación de sets	9
2	<i>Problema 2</i>	10
2.1	Enunciado y Supuestos	10
2.1.1	Enunciado	10
2.1.2	Supuestos	10
2.2	Variables del Modelo	11
2.2.1	Variables	11
2.3	Modelo de programación lineal	11
2.4	Solución	12
2.5	Informe	12
3	<i>Problema 3</i>	13
3.1	Enunciado y Supuestos	13
3.1.1	Enunciado	13
3.1.2	Supuestos	13
3.2	Diseño del Algoritmo	13
3.3	Adaptación a una Red de Flujo y Significado de la Salida	14
3.4	Pseudocódigo y Estructuras de Datos Utilizadas	15
3.5	Ejemplo de Seguimiento con Datos Reducidos	16
3.6	Diagrama 1: Red de Flujo (Construcción Inicial)	17
3.7	Diagrama 2: Red de Flujo con Flujo Asignado (Solución Óptima)	17
3.8	Interpretación del Ejemplo	17
3.9	Conclusión	18
3.10	Análisis de Complejidad	18
3.11	Función <code>Asignar_Respaldo</code>	18
3.12	Conclusión	19
3.13	Generación de sets	19
3.14	Análisis de complejidad y resultados	19
3.15	Conclusión de análisis de complejidad	20
4	Referencias	20

1 ***Problema 1***

1.1 Enunciado y Supuestos

1.1.1 Enunciado

- Cualquier cadena puede ser descompuesta en secuencias de palíndromos. Por ejemplo, la cadena ARACALACANA se puede descomponer de las siguientes formas: ARA CALAC ANA
ARA C ALA C ANA
A R A CALAC A N A
etc.

Desarrollar un algoritmo de programación dinámica que encuentre el menor número de palíndromos que forman una cadena dada. Por ejemplo, para ARACALACANA debería devolver 3.

1.1.2 Supuestos

- Se supone que el algoritmo recibe una cadena (string), con una frase, y el algoritmo buscara la mínima cantidad de palíndromos posibles que forma la cadena dada por parámetro, mediante programación dinámica.
- La frase, si tiene espacios o caracteres especiales, van a ser tratados como letras, ejemplo A!NEUQUEN!A, va a devolver 1, ya que se es indiferente con los caracteres y letras.

1.2 Diseño

- La ecuación de recurrencia a la que se se llega es: $opt[i] = 1$ si $palabra[i:]$ es palíndromo y $opt[i] = \min(opt[j+1] + 1)$ para todo j tal que $palabra[i:j+1]$ es palíndromo.

- Subestructura óptima: La solución óptima para el prefijo $palabra[0:i]$ depende de las soluciones óptimas de los sufijos $palabra[j+1:]$, que son subproblemas del problema original.

Subproblemas superpuestos: Evaluar si una subcadena es palíndromo se repite muchas veces. Por eso se usa una tabla booleana `es_palindromo[i][j]` para memorizar los resultados.

- Se usa memoization en 2 estructuras de datos, para guardar en un arreglo de enteros (todos infinitos positivos de base) $opt[i]$ el mínimo numero de palíndromos para $palabra[i:]$. Y para guardar en una matriz booleana (con todas las posiciones false de base) de $n \times n$ `es_palindromo[i][j]` si $palabra[i:j+1]$ es un palíndromo.
- Las estructuras de datos utilizadas para esta resolución son una lista de enteros, y una matriz de booleanos, siendo `opt` la lista y `es_palindromo` la matriz.

1.3 Pseudocódigo

```
1 FUNCION min_palindromos(palabra):
2   n = largo(palabra)
3
4   optimo = [float('infinito')] * (n + 1)
5   opt[n] = 0
6
7   es_palindromo = [[False] * n for _ in range(n)]
8
9   Para i desde n - 1 hasta 0 con paso -1 hacer
10      Para j desde i hasta n - 1 hacer
11         Si palabra[i] = palabra[j] y (j - i <= 1 o
es_palindromo[i + 1][j - 1]) entonces
12             es_palindromo[i][j] <- Verdadero
13         FinSi
14      FinPara
15   FinPara
16
17   Para i desde n - 1 hasta 0 con paso -1 hacer
18      Para j desde i hasta n - 1 hacer
19         Si es_palindromo[i][j] entonces
20             opt[i] <- minimo entre opt[i] y (1 + opt[j +
1])
21         FinSi
22      FinPara
23   FinPara
24
25   DEVOLVER opt[0]
26
27
28 cadena = "ALLAERRESAGASRADAR"
29 print(min_palindromos(cadena))
30
```

1.4 Análisis de Complejidad

- Los puntos clave para analizar la complejidad de este algoritmo son los 2 for anidados que hay tanto para el procesamiento de los palíndromos que se ejecuta para cada par i, j en:

```
1     for i in range(n - 1, -1, -1):
2         for j in range(i, n):
3             if palabra[i] == palabra[j] and (j - i <= 1 or
es_palindromo[i + 1][j - 1]):
4                 es_palindromo[i][j] = True
5
```

Y para la construcción de la tabla booleana $opt[i][j]$ que también revisa cada subcadena $palabra[i : j + 1]$, y si la misma es palíndromo realiza una operación constante en:

```
1     for i in range(n - 1, -1, -1):
2         for j in range(i, n):
3             if es_palindromo[i][j]:
4                 opt[i] = min(opt[i], 1 + opt[j + 1])
5
```

Por lo tanto, en ambos for anidados, el peor caso implica una doble iteración sobre n , por ende la complejidad temporal total del algoritmo es de $O(N^2)$, siendo N el largo de la cadena.

1.5 Seguimiento

- Vamos a hacer el seguimiento para el caso de la cadena "ARACALACANA": lo primero que hace el algoritmo es obtener el largo de la cadena recibida mediante `len()`, normalizar todo a mayúsculas y procede a generar las 2 estructuras utilizadas, la lista de enteros y la matriz de booleanos, para luego llegar a la primer parte importante, el preprocesamiento de los , donde se marca como True toda subcadena de `palabra[i : j + 1]` que sea un palíndromo.

`n = largo(palabra) = 11`

`opt = [inf, inf, ..., inf, 0]` (longitud 12)

`es_palindromo[i][j] = False` para toda i, j

- Para ver si un substring es palíndromo o no, se utilizan 3 factores, si `palabra[i] = palabra[j]` (la palabra empieza y termina con la misma letra) y si $j - i \leq 1$ (es de 1 o 2 caracteres) o `es_palindromo[i+1][j-1] = True`. Entonces empieza desde $i = 10$ y siempre lo primero que llena en cada i es la diagonal, que es trivialmente un palíndromo, al ser un solo carácter, pero empieza llenando la matriz desde el final.
- La primer posición analizada es `[10][10]`, siendo la cadena "A", que es un caso trivial, cumpliendo la condición y llenando la tabla en esa posición con True, luego sigue `[9][9]`, que ocurre lo mismo que en el caso anterior, siendo simplemente el carácter "N", `[9][10]` no cumple la condición, ya que no empieza y termina con la misma letra, quedándose en False..., saltamos al ejemplo del siguiente palíndromo no trivial, que es `[8][10]` la cadena "ANA" cumple que empieza y termina con la misma letra, no tiene menos de 2 caracteres, pero `es_palindromo[i+1][j-1]` que en este caso es `[9][9]` es True, entonces rellena True dándola como palíndromo.
- Para el final de las iteraciones el algoritmo rellena la tabla ubicando True en todos los palíndromos que encuentra.

	A	R	A	C	A	L	A	C	A	N	A
A	T	F	T	F	F	F	F	F	F	F	F
R	F	T	F	F	F	F	F	F	F	F	F
A	F	F	T	F	T	F	F	F	T	F	F
C	F	F	F	T	F	F	F	T	F	F	F
A	F	F	F	F	T	F	T	F	F	F	F
L	F	F	F	F	F	T	F	F	F	F	F
A	F	F	F	F	F	F	T	F	T	F	F
C	F	F	F	F	F	F	F	T	F	F	F
A	F	F	F	F	F	F	F	F	T	F	T
N	F	F	F	F	F	F	F	F	F	T	F
A	F	F	F	F	F	F	F	F	F	F	T

Figure 1: Representación de la matriz de palíndromos.

- Una vez rellena la tabla, pasa a la construcción de $\text{opt}[i]$ buscando la mínima cantidad de palíndromos de la palabra, donde $\text{opt}[n] = 0$, siendo el caso base la string vacía, luego empieza la iteración desde $i = 10$ hasta 0.
- Entonces empieza con $[10][10]$, le pregunta a la tabla si es palíndromo, en este caso lo es, por ende $\text{opt}[i]$ es igual al mínimo entre si mismo, que de base es infinito, y $1 + \text{opt}[j+1]$, que en este caso es $1 + \text{opt}[11]$ (que es 0), entonces $\text{opt}[10] = 1$. Sigue con $i = 9, j = 9$, se le pregunta si la $[9][9]$ es True, lo cual es correcto, entonces se usa la ecuación siendo $\text{opt}[9] = \min(\text{opt}[9], 1 + \text{opt}[10])$, que termina siendo $\text{opt}[9] = \min(\text{infinito}, 2)$, $\text{opt}[9] = 2$, y así sigue con toda la tabla para finalmente encontrar el optimo que es 3.

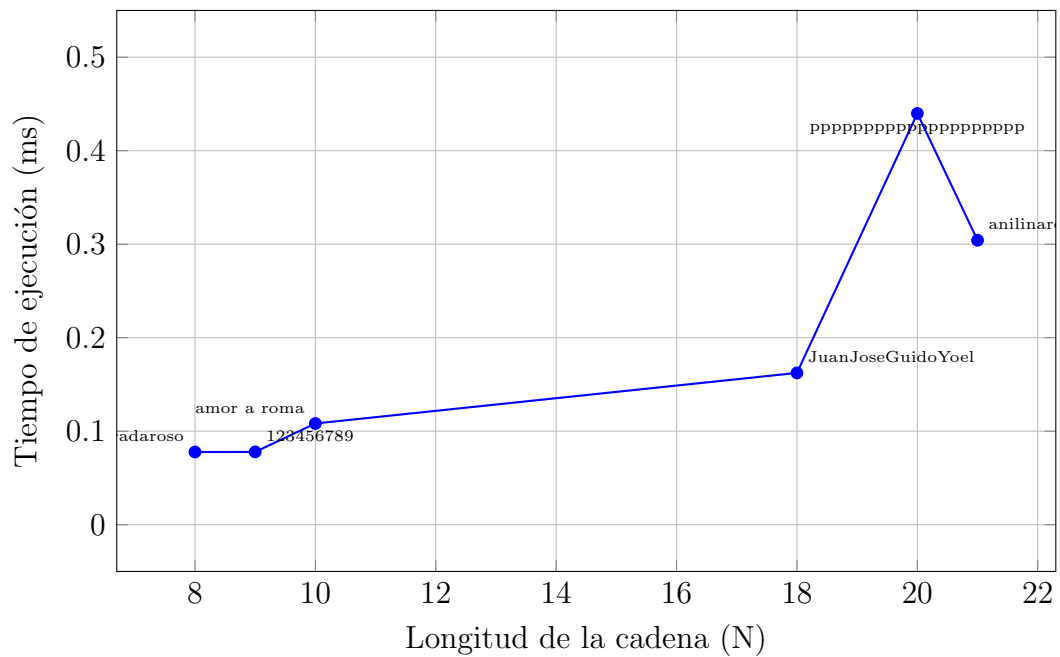
1.6 Casos de prueba y mediciones

Entrada	Cantidad de palíndromos	Tiempo (ms)
radaroso	2	0.077725
123456789	9	0.077863
amor a roma	1	0.108240
JuanJoseGuidoYoel	15	0.162336
pppppppppppppppppppppp	1	0.439882
anilinareconocersalas	3	0.304222

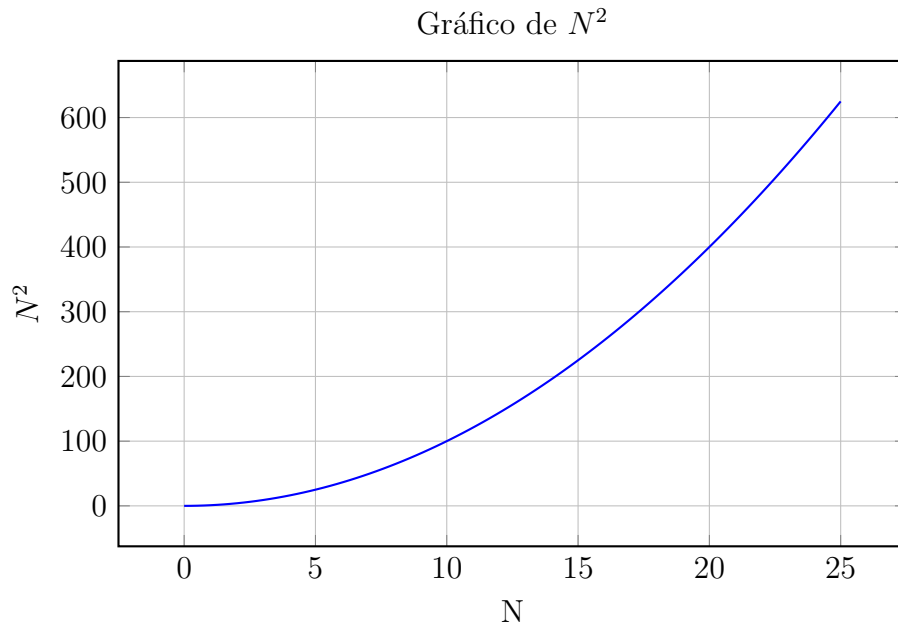
1.7 Resultados

Tiempo de ejecución vs longitud de la cadena (N)

Relación entre longitud N y tiempo de ejecución



Curva de N^2



1.8 Conclusiones

- Se ve que los sets cumplen con la complejidad prevista, exceptuando el caso de las p, donde se deja ver que las operaciones descartadas por ser menores a N^2 afectan a la complejidad, exceptuando este caso, se ve que a medida que aumenta el largo, el tiempo sube exponencialmente, cumpliendo lo previsto.

1.9 Generación de sets

- Los sets fueron generados arbitrariamente, para poder probar casos como distinción entre minúscula y mayúscula, espacios, ningún palíndromo, palíndromos encadenados, y una cadena de un solo carácter.

2 *Problema 2*

2.1 Enunciado y Supuestos

2.1.1 Enunciado

- Concesiones Argentina 2000 SRL tiene la concesión de los espacios publicitarios en las paradas de colectivos de un municipio. Son en total 200 paradas y tiene ofertas de distintos productos para el próximo mes:
- Cliente A ofrece USD 50000 por ocupar 30 paradas
- Cliente B ofrece USD 100000 por ocupar 80 paradas o
- USD 120000 por 120 paradas (sólo una de ambas opciones)
- Cliente C ofrece USD 100000 por ocupar 75 paradas
- Cliente D ofrece USD 80000 por ocupar 50 paradas
- Cliente E ofrece USD 5000 por ocupar 2 paradas
- Cliente F ofrece USD 40000 por ocupar 20 paradas
- Cliente G ofrece USD 90000 por ocupar 100 paradas
- Por ser competidores directos, no se puede hacer publicidad simultáneamente de los clientes A y D. Se desea determinar a qué clientes se concesionará la publicidad de las paradas para maximizar el beneficio total.

2.1.2 Supuestos

Para el planteamiento del modelo se asumen las siguientes premisas:

- El entorno es determinístico; los datos de entrada se conocen exactamente sin error.
- Las relaciones entre variables son lineales, por lo que cualquier término no lineal se desprecia.
- Los recursos y demandas involucrados están completamente definidos y son medibles, permitiendo una modelación precisa.
- Se asume que los costos y beneficios pueden expresarse como funciones lineales de las variables de decisión.
- No se consideran incertidumbres, variaciones estocásticas ni fenómenos externos que puedan afectar la solución.

2.2 Variables del Modelo

En el modelo se definen las siguientes variables:

2.2.1 Variables

las variables para resolver el problema son las siguientes:

- a = incluir cliente A (variable binaria)
- b_1 = incluir cliente B de 80 paradas (variable binaria)
- b_2 = incluir cliente B de 120 paradas (variable binaria)
- c = incluir cliente C (variable binaria)
- d = incluir cliente D (variable binaria)
- e = incluir cliente e (variable binaria)
- f = incluir cliente f (variable binaria)
- g = incluir cliente G (variable binaria)

2.3 Modelo de programación lineal

La función objetivo a maximizar es la siguiente:

$$z = 50000a + 100000b_1 + 120000b_2 + 100000c + 80000d + 5000e + 40000f + 90000g$$

$\max(z)$

Restricciones del modelo:

$$\begin{aligned} 30a + 80b_1 + 120b_2 + 75c + 50d + 2e + 20f + 100g &\leq 200 \\ b_1 + b_2 &\leq 1 \\ a + d &\leq 1 \end{aligned}$$

- La primera ecuación nos indica el total de paradas de se concesionaria a los clientes para la publicidad no supere las 200 paradas disponibles.
- La segunda ecuación nos indica que de las ambas opciones que tiene el cliente B solo pueda ser 1 y no las 2.
- La tercera ecuación nos indica que no se puede hacer publicidad simultáneamente de los cliente A y D.

2.4 Solución

Para resolver el modelo se desarrollo un programa utilizando Python y PuLP, el cual es el siguiente:

```
import pulp

def resolver_concesiones():
    paradas=200
    problema = pulp.LpProblem("Concesiones_Argentina_2000_SRL", pulp.LpMaximize)
    a = pulp.LpVariable("a", cat="Binary")
    b_1 = pulp.LpVariable("b_1", cat="Binary")
    b_2 = pulp.LpVariable("b_2", cat="Binary")
    c = pulp.LpVariable("c", cat="Binary")
    d = pulp.LpVariable("d", cat="Binary")
    e = pulp.LpVariable("e", cat="Binary")
    f = pulp.LpVariable("f", cat="Binary")
    g = pulp.LpVariable("g", cat="Binary")

    problema += (50000*a + 100000*b_1 + 120000*b_2 + 100000*c + 80000*d + 5000*e + 40000*f + 90000*g)
    problema += (30*a + 80*b_1 + 120*b_2 + 75*c + 50*d + 2*e + 20*f + 100*g) <= paradas
    problema += (a + d <= 1)
    problema += (b_1 + b_2 <= 1)

    problema.solve()

    return problema

if __name__ == "__main__":
    problema = resolver_concesiones()
    print("Estado:", pulp.LpStatus[problema.status])
    for var in problema.variables():
        print(f"{var.name}: {int(var.varValue)}")
    print("Ganancia total:", pulp.value(problema.objective))
```

Como resultado dio que para maximizar la ganancia tendríamos que escoger a los clientes A, B_1(la opción de 80 paradas), C y E.

2.5 Informe

La solución del modelo de programación lineal nos propone que para determinar a qué clientes se concesionará la publicidad de las paradas para maximizar el beneficio total son las siguientes:

- Cliente A ofrece USD 50000 por ocupar 30 paradas
- Cliente B ofrece USD 100000 por ocupar 80 paradas
- Cliente C ofrece USD 100000 por ocupar 75 paradas
- Cliente E ofrece USD 5000 por ocupar 2 paradas

Con estos resultados nos queda una ganancia de USD 255000 y la cantidad de paradas utilizadas para la publicidad son de 187, la cantidad de paradas no llega al limite pero tampoco se puede utilizar para darles publicidad a otros clientes porque las paradas requeridas por los otros clientes es mayor a la disponible.

3 *Problema 3*

3.1 Enunciado y Supuestos

3.1.1 Enunciado

- Estamos construyendo una red WAN con n antenas y queremos que tenga un buen nivel de tolerancia a fallas. Dada una antena, su conjunto de backup de tamaño k es el conjunto de k antenas que se encuentran a una distancia menor a D . Queremos evitar que una antena pertenezca al conjunto de backup de más de b antenas, precisamente para evitar que un fallo pueda afectar a una porción importante de la red. Suponer que conocemos los valores D , b y k , y que tenemos una matriz $d[1..n, 1..n]$ con las distancias entre antenas, de forma tal que $d[i,j]$ es la distancia entre la antena i y la j .

Plantear un algoritmo de complejidad polinomial que encuentre el conjunto de backup de tamaño k de cada una de las n antenas, de forma tal que ninguna aparezca en más de b conjuntos de backup, o bien, que indique que no existe una solución posible. Debe apoyarse en el algoritmo estándar de Ford-Fulkerson, la variante escalada o la de Edmonds-Karp.

3.1.2 Supuestos

- 1. **Datos de Entrada:** Se dispone de una matriz de distancias $d[1 \dots n, 1 \dots n]$ donde $d[i, j]$ es la distancia entre la antena i y la j . Se asume que dicha matriz está completa, es decir, se conocen todas las distancias relevantes para la red.
- 2. **Distancia Umbral:** Solo se consideran como candidatos backup aquellas antenas que se encuentran a una distancia menor a D respecto a la antena en cuestión.
- 3. **Tamaño del Conjunto de Backup:** Para cada antena se debe seleccionar exactamente un conjunto de backup de tamaño k .
- 4. **Restricción de Uso:** Una antena puede formar parte del backup de, a lo sumo, b otras antenas. Esto previene la sobrecarga y, en caso de fallo, evita que el problema se propague a una parte importante de la red.
- 5. **No Auto-backup:** Se asume que una antena no puede ser backup de sí misma (es decir, no existen bucles desde un nodo a sí mismo en la red de flujo).
- 6. **Linealidad y Exactitud:** Se supone que los datos son exactos y que las relaciones (por ejemplo, “estar a menos de D ”) se definen de forma lineal y determinística.

3.2 Diseño del Algoritmo

El algoritmo se basa en la modelación del problema vía una red de flujo, y se resuelve mediante un algoritmo clásico (por ejemplo, Ford-Fulkerson, su variante

escalada o Edmonds-Karp). A continuación se describen los pasos para adaptar los datos de entrada a una red de flujo y cómo interpretar la salida.

3.3 Adaptación a una Red de Flujo y Significado de la Salida

Construcción de la Red de Flujo: Se genera un grafo dirigido que transformará el problema de asignación de backups en un problema de flujo:

a. **Nodos:**

- Se crea un nodo fuente S y un nodo sumidero T .
- Se representan las antenas en dos copias: la parte izquierda (o de origen) y la parte derecha (o de destino). La copia de la izquierda representa a cada antena que debe elegir k backups; la copia de la derecha representa a cada antena como potencial backup, con su límite de participación.

b. **Aristas desde la Fuente:** Se crea un arco desde S a cada antena (en la copia izquierda) con capacidad k . Esto asegura que cada antena debe “enviar” k unidades de flujo, interpretadas como la cantidad de backups necesarios.

c. **Aristas Intermedios:** Se conecta una antena i (de la copia izquierda) con una antena j (en la copia derecha) mediante un arco de capacidad 1 si y sólo si $d[i, j] < D$. Esta condición garantiza que únicamente se consideren antenas cercanas (según el umbral D).

d. **Aristas al Sumidero:** Cada antena j (de la copia derecha) se conecta con el sumidero T mediante un arco de capacidad b . Esto impone que ninguna antena sea asignada como backup a más de b antenas.

Interpretación de la Salida: Una vez construida la red, se ejecuta el algoritmo de flujo, Edmonds-Karp. Si el flujo máximo encontrado es exactamente $n \cdot k$ (ya que cada una de las n antenas debe enviar k unidades de flujo), entonces:

- La solución asigna, para cada antena i , un conjunto de $\{j \mid \text{flujo en el arco } (i, j) = 1\}$ de tamaño k .
- Cada antena j aparecerá en el conjunto de backup de al menos 0 y como máximo b antenas, dado el límite impuesto en el arco de j a T .

Si el flujo máximo es menor que $n \cdot k$, se concluye que no es posible asignar respaldos cumpliendo todas las restricciones.

Diagrama de la Red de Flujo: A continuación se muestra un diagrama esquemático de la modelación:

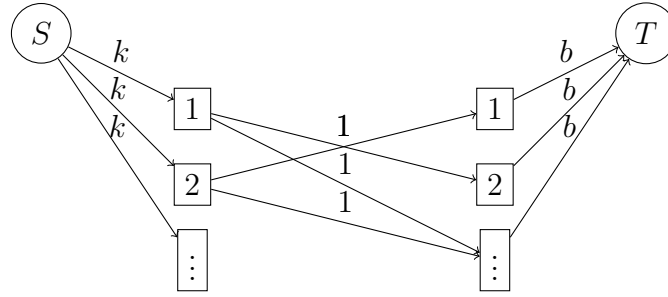


Figure 2: Modelo de Red de Flujo para la Asignación de Backups

3.4 Pseudocódigo y Estructuras de Datos Utilizadas

A continuación se presenta un pseudocódigo que resume el proceso completo:

```

1 FUNCION Construir_Grafo(distancias, D, b, k):
2   n <-- longitud(distancias)
3   Crear grafo dirigido G vacio
4   fuente <-- "S"
5   sumidero <-- "T"
6
7   Agregar nodo fuente a G
8   Agregar nodo sumidero a G
9
10  Para i desde 0 hasta n-1 hacer:
11    entrada <-- "in_" concatenado con i
12    salida <-- "out_" concatenado con i
13    Agregar nodo "entrada" a G
14    Agregar nodo "salida" a G
15    Agregar arco (fuente, entrada) con capacidad k a G
16    Agregar arco (salida, sumidero) con capacidad b a G
17
18  Para i desde 0 hasta n-1 hacer:
19    Para j desde 0 hasta n-1 hacer:
20      Si (i != j) y (distancias[i][j] < D) entonces:
21        Agregar arco ("in_" concatenado con i, "out_"
concatenado con j)
22          con capacidad 1 a G
23  Retornar G
24
25 FUNCION Asignar_Respaldo(distancias, D, b, k):
26   G <-- Construir_Grafo(distancias, D, b, k)
27   (flujo_total, flujo) <-- CalcularFlujoMaximo(G, "S", "T")
28   n <-- longitud(distancias)
29
30   Si flujo_total < n * k entonces:
31     Retornar "No existe solucion"
32
33   resultado <-- arreglo de n listas vacias
34
35   Para i desde 0 hasta n-1 hacer:
36     entrada <-- "in_" concatenado con i
37     Para j desde 0 hasta n-1 hacer:
38       Si (i != j) y (distancias[i][j] < D) entonces:
39         salida <-- "out_" concatenado con j

```

```

40         Si flujo[entrada][salida] = 1 entonces:
41             Agregar j a resultado[i]
42     Imprimir resultado
43     Retornar resultado

```

Listing 1: Pseudocódigo para construir el grafo y asignar respaldo

Estructuras de Datos:

- **Grafo G :** Representado mediante listas de adyacencia (o matriz de adyacencia) donde cada arco almacena su capacidad y flujo actual.
- **Nodos:** Cada antenna se divide en dos nodos (u_i y v_i) para representar su rol como emisor y receptor en la red.
- **Cola:** En la implementación de Edmonds-Karp se utiliza una cola para el recorrido en anchura (BFS) en búsqueda de caminos aumentantes.
- **Aristas:** Cada arco se representa como una estructura o tupla con información sobre el nodo origen, nodo destino, capacidad máxima y flujo actual.

3.5 Ejemplo de Seguimiento con Datos Reducidos

Consideremos el siguiente set reducido de datos:

- Número de antenas: $n = 3$.
- Valor de umbral de distancia: $D = 2$.
- Tamaño de respaldo requerido: $k = 1$ (cada antenna necesita un backup).
- Límite de apariciones como backup: $b = 1$ (cada antenna puede servir como backup para, a lo sumo, una otra antenna).
- Matriz de distancias:

$$\begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

Dado que $d[i][j] = 1 < D$ para $i \neq j$, cada antenna i podrá conectarse a las otras dos antenas j .

Sin embargo, considerando las restricciones de capacidad:

- Cada antenna debe suministrar $k = 1$ unidad al conjunto de backup.
- Cada antenna (como backup) puede recibir como máximo $b = 1$ unidad.

Se busca una asignación en la que la solución sea de la forma:

Backup de la antenna 0 es 1, Backup de la antenna 1 es 2, Backup de la antenna 2 es 0.

3.6 Diagrama 1: Red de Flujo (Construcción Inicial)

El siguiente diagrama muestra la red de flujo construida a partir de los datos:

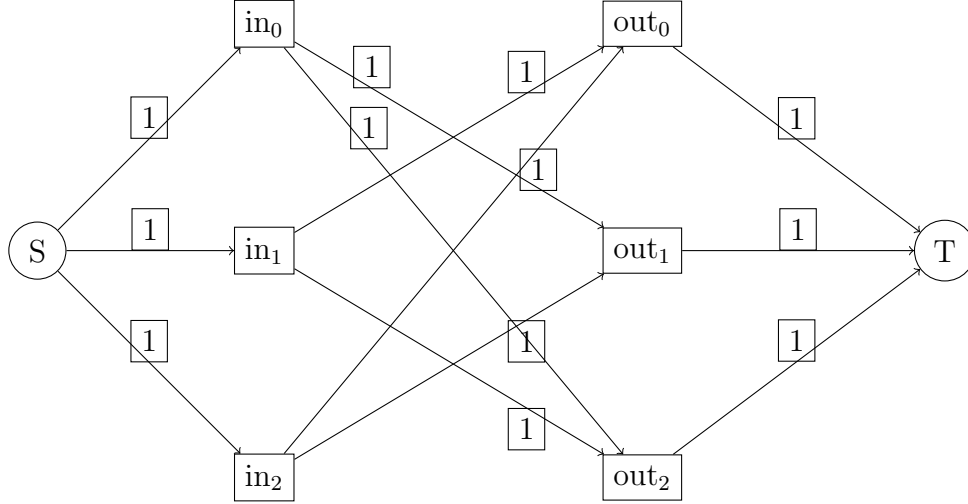


Figure 3: Red de flujo inicial construida a partir de los datos reducidos.

3.7 Diagrama 2: Red de Flujo con Flujo Asignado (Solución Óptima)

Suponiendo que el algoritmo de flujo encuentra la siguiente asignación:

$$\text{in}_0 \rightarrow \text{out}_1$$

$$\text{in}_1 \rightarrow \text{out}_2$$

$$\text{in}_2 \rightarrow \text{out}_0$$

el diagrama resaltado a continuación muestra la solución óptima (se dibujan los arista seleccionados en rojo y gruesos):

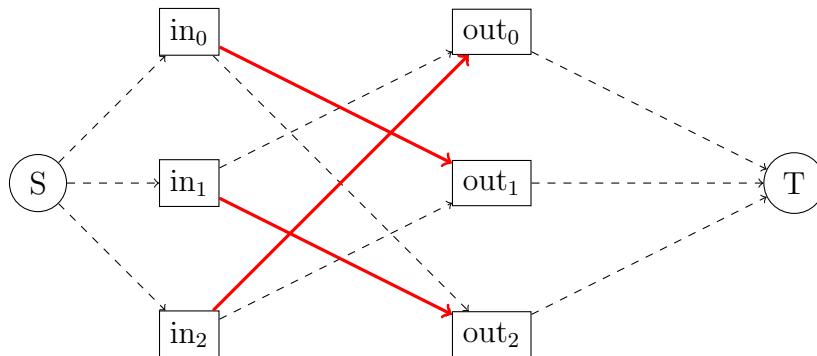


Figure 4: Red de flujo con la asignación óptima: antena 0 respalda a 1, 1 respalda a 2 y 2 respalda a 0.

3.8 Interpretación del Ejemplo

Con la solución óptima obtenida:

- La antena 0 utiliza la conexión `in_0 → out_1` para asignar a la antena 1 como backup.
- La antena 1 utiliza `in_1 → out_2` para asignar a la antena 2.
- La antena 2 utiliza `in_2 → out_0` para asignar a la antena 0.

El flujo total alcanzado es igual a $n \times k = 3 \times 1 = 3$, lo que indica que se cumple la restricción y se ha encontrado una solución válida. Si el flujo total hubiese sido menor, el algoritmo habría informado que no existe solución posible bajo las restricciones D , b y k definidas.

3.9 Conclusión

El ejemplo implementado con un set de datos reducido ilustra cómo se transforma el problema de asignación de respaldos en una red de flujo, utilizando la división en nodos de entrada y salida y estableciendo capacidades según los parámetros k y b .

3.10 Análisis de Complejidad

Función Construir_Grafo

- Obtener la longitud de la matriz de distancias.
 $\Rightarrow \mathcal{O}(1)$
- Creación de nodos especiales (S , T) y el grafo vacío.
 $\Rightarrow \mathcal{O}(1)$
- Bucle sobre n nodos para crear nodos de entrada/salida y conectar fuente y sumidero.
Cada iteración realiza operaciones constantes. Total: $\mathcal{O}(n)$
- Doble bucle anidado para agregar aristas de respaldo entre pares de nodos.
Hay $n(n-1)$ comparaciones posibles, cada una con operaciones constantes.
 $\Rightarrow \mathcal{O}(n^2)$

Complejidad total de Construir_Grafo:

$$\boxed{\mathcal{O}(n^2)}$$

3.11 Función Asignar_Respaldo

- Llamada a `Construir_Grafo`. Ya sabemos que es $\mathcal{O}(n^2)$
- Cálculo de flujo máximo en un grafo de tamaño:
 - Nodos: $2n + 2$ (entrada/salida por nodo, fuente y sumidero)
 - Aristas: Hasta $2n$ (fuente y sumidero) + $n(n-1)$ (entre entradas y salidas)

- Usando el algoritmo de Edmonds-Karp (por defecto en NetworkX):

$$\mathcal{O}(V \cdot E^2) = \mathcal{O}(n \cdot (n^2)^2) = \mathcal{O}(n^5)$$

- Obtener longitud de matriz: $\mathcal{O}(1)$
- Comparación: $\mathcal{O}(1)$
- Inicializar arreglo de n listas vacías: $\mathcal{O}(n)$
- Doble bucle sobre nodos ($n \times n$) y acceso al flujo:
 - Comparaciones y accesos al diccionario de flujo: operaciones constantes.
 - Total: $\mathcal{O}(n^2)$
- imprimir resultado: $\mathcal{O}(n^2)$

Complejidad total de Asignar_Respaldo:

$$\boxed{\mathcal{O}(n^5)}$$

3.12 Conclusión

A pesar de haber realizado el gráfico y las operaciones necesarias en una complejidad menor que N^2 , la complejidad se ve afectada por el uso de Edmonds-Karp.

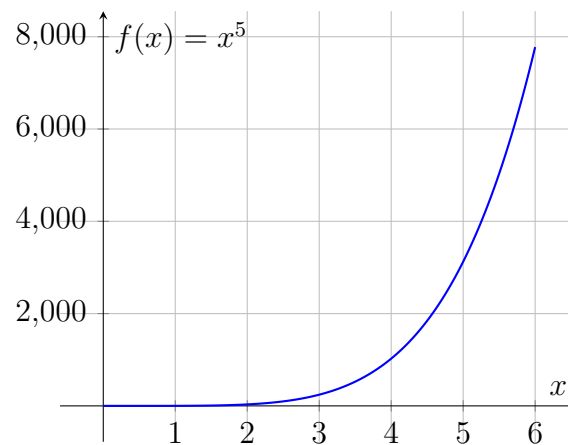
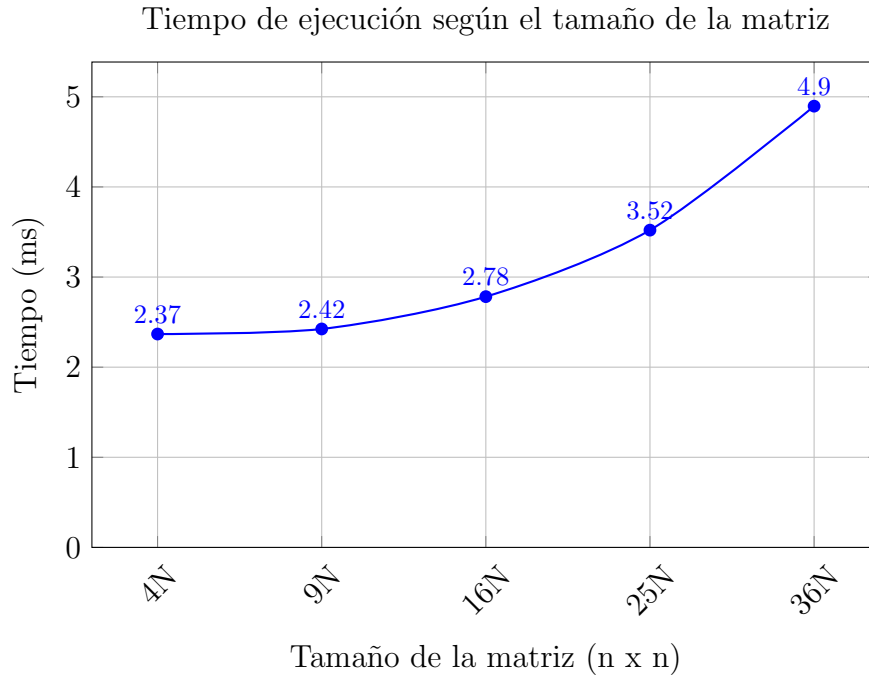
3.13 Generación de sets

- Los sets fueron generados arbitrariamente, para poder probar casos con una matriz cada vez mas grande.

3.14 Análisis de complejidad y resultados

Matriz $n \times n$	Tiempo (ms)
4N	2.3677
9N	2.4239
16N	2.7826
25N	3.5207
36N	4.8964

Table 1: Tiempos de ejecución (en milisegundos) para diferentes tamaños de matriz.

Figure 5: Gráfico de la función x^5 .

3.15 Conclusión de análisis de complejidad

Los resultados muestran que tiende a cumplir con la curva esperada, a pesar de no ser tan exponencial su escalada en comparación con los datos ingresados.

4 Referencias

- Edmonds, J., & Karp, R. M. (1972). Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2), 248–264. <https://doi.org/10.1145/321607.321609>

- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). *Exploring network structure, dynamics, and function using NetworkX*. In *Proceedings of the 7th Python in Science Conference (SciPy2008)* (pp. 11–15). SciPy Conference.
- Mitchell, S. (n.d.). *PuLP: A linear programming modeler written in Python* [Computer software]. Recuperado el 1 de octubre de 2025, de <https://coin-or.github.io/pulp/>